

Modelos Generativos Profundos

Clase 8: Modelos autorregresivos y LLMs (parte IV)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

BERT

Modelos encoder-decoder

Vision Transformer

CLIP

Chameleon

BERT

Bidirectional encoder representations from transformers

BERT es un modelo tipo Transformer usado para tareas de comprensión del lenguaje.

- La entrada al modelo puede ser una secuencia de tokens $S \in \mathcal{V}^*$ o dos secuencias de tokens $S_1, S_2 \in \mathcal{V}^*$ concatenadas.
- No sigue un enfoque autorregresivo, por lo que no necesita masking causal y no puede generar texto.
- Los embeddings contextuales obtenidos con BERT pueden ser usados para otras tareas. También es posible obtener un embedding representante de toda la secuencia de entrada.
- BERT utiliza 3 tokens especiales: **<MASK>**, **<CLS>** y **<SEP>**.

Tokenización

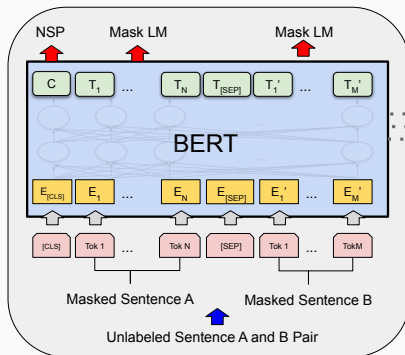
- BERT añade el token <CLS> al comienzo. Su embedding contextual es usado como la representación agregada de la entrada.
- Las secuencias $S_1, S_2 \in \mathcal{V}^*$ concatenadas son separadas por el token <SEP>, y cada secuencia incluye un embedding adicional:

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[\text{CLS}]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[\text{SEP}]}$	E_{he}	E_{likes}	E_{play}	$E_{\text{##ing}}$	$E_{[\text{SEP}]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

Entrenamiento

BERT se entrena mediante dos tareas autosupervisadas:

- **Masked language model:** se ocultan tokens en la secuencia de entrada (se cambian por <MASK>) y el modelo debe predecirlos.
- **Next sentence prediction:** el modelo debe predecir si el texto $S_2 \in \mathcal{V}^*$ es la continuación del texto $S_1 \in \mathcal{V}^*$.



Entrenamiento

Dada una secuencia $S = (w_1, \dots, w_T) \in \mathcal{V}^*$, si $\tilde{S}$ es su versión enmascarada en las posiciones $\mathcal{M}(S) \subset \{1, \dots, T\}$:

$$L_{\text{MLM}}(S) = - \sum_{t \in \mathcal{M}(S)} \log p_{\theta}(w_t | \tilde{S})$$

Dado un par de secuencias $S_1, S_2 \in \mathcal{V}^*$ y su etiqueta de clasificación binaria $y \in \{\text{IsNext}, \text{NotNext}\}$:

$$L_{\text{NSP}}(S_1, S_2) = - \log p_{\theta}(y | S_1, S_2)$$

Finalmente, BERT es entrenado con la siguiente función de pérdida combinada:

$$L_{\text{BERT}} = \frac{1}{|\mathcal{D}_{\text{MLM}}|} \sum_{S \in \mathcal{D}_{\text{MLM}}} L_{\text{MLM}}(S) + \frac{1}{|\mathcal{D}_{\text{NSP}}|} \sum_{(S_1, S_2) \in \mathcal{D}_{\text{NSP}}} L_{\text{NSP}}(S_1, S_2)$$

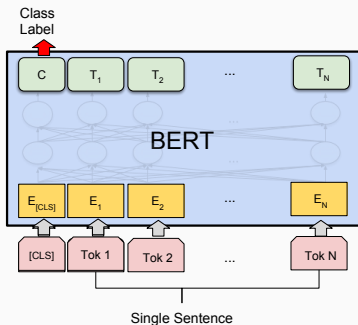
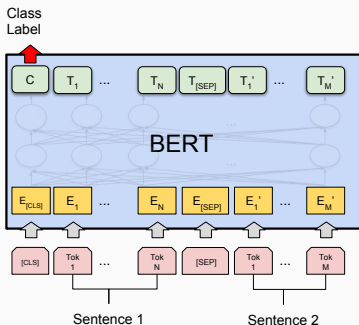
Una vez entrenado, BERT se puede utilizar para distintas tareas:

- Como modelo de embedding contextual para cada token o para toda la secuencia de entrada.
- Para clasificación de secuencias o pares de secuencias.
- Para la tarea de span extraction.

Uso como clasificador

Para clasificar una secuencia o un par de secuencias, se puede utilizar el embedding contextual del token <CLS>.

En ambos casos es necesario cambiar el cabezal de clasificación original por otro cabezal específico para la tarea y hacer **fine tuning**.



Uso para extracción de subsecuencias

Para la tarea de span extraction:

- Secuencia de pregunta: $S_{\text{query}} \in \mathcal{V}^*$.
- Secuencia de contexto: $S_{\text{context}} \in \mathcal{V}^*$.
- Secuencia concatenada: $S_{\text{concat}} = S_{\text{query}} \langle \text{SEP} \rangle S_{\text{context}} \in \mathcal{V}^*$.
- Predicción esperada: $S_{\text{span}} = (w_{t_{\text{start}}}, \dots, w_{t_{\text{end}}}) \subset S_{\text{context}}$.

Para este par pregunta-respuesta, la función de pérdida es:

$$L_{\text{span}}(S_{\text{concat}}) = -\log p_{\theta}(t_{\text{start}} | S_{\text{concat}}) - \log p_{\theta}(t_{\text{end}} | S_{\text{concat}}),$$

y el fine tuning se realiza minimizando esta función de pérdida sobre un conjunto de pares pregunta-respuesta:

$$L_{\text{span}} = \frac{1}{|\mathcal{D}_{\text{span}}|} \sum_{(S_{\text{query}}, S_{\text{context}}, S_{\text{span}}) \in \mathcal{D}_{\text{span}}} L_{\text{span}}(S_{\text{concat}})$$

Uso para extracción de subsecuencias

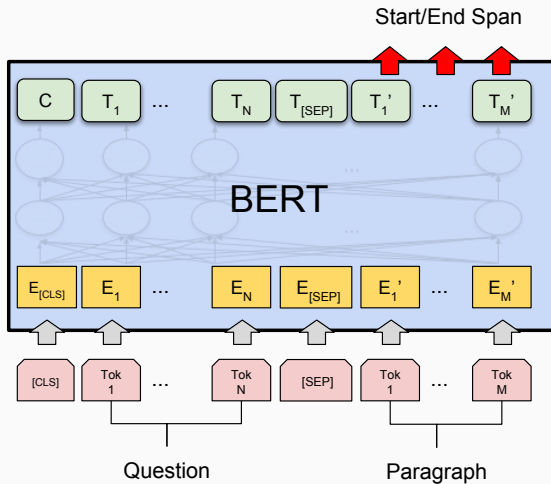
Luego, en inferencia, dado un par $S_{\text{query}}, S_{\text{context}} \in \mathcal{V}^*$, se obtiene la secuencia concatenada $S_{\text{concat}} = S_{\text{query}}\langle\text{SEP}\rangle S_{\text{context}}$ y se predicen los índices $t_{\text{start}}, t_{\text{end}} \in \{1, \dots, |S_{\text{context}}|\}$ con mayor probabilidad:

$$(t_{\text{start}}, t_{\text{end}}) = \underset{\substack{t_{\text{start}}, t_{\text{end}} \in \{1, \dots, |S_{\text{context}}|\} \\ t_{\text{start}} \leq t_{\text{end}}}}{\text{argmax}} \log p_{\theta}(t_{\text{start}}, t_{\text{end}} | S_{\text{concat}}), \text{ donde}$$

$$\log p_{\theta}(t_{\text{start}}, t_{\text{end}} | S_{\text{concat}}) = \log p_{\theta}(t_{\text{start}} | S_{\text{concat}}) + \log p_{\theta}(t_{\text{end}} | S_{\text{concat}}),$$

y se devuelve el span $S_{\text{span}} = (w_{t_{\text{start}}}, \dots, w_{t_{\text{end}}}) \subset S_{\text{context}}$ como respuesta a la pregunta S_{query} .

Uso para extracción de subsecuencias

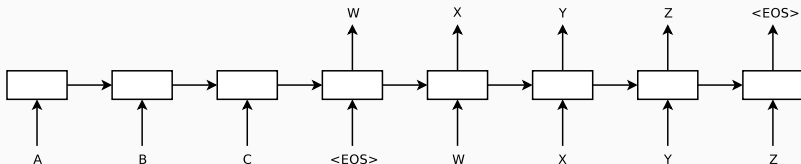


Modelos encoder-decoder

Modelo seq2seq

Si se tiene un modelo autorregresivo condicional $p(S|S_{\text{contexto}})$, donde $S_{\text{contexto}} \in \mathcal{V}^*$ es una secuencia condicional, entonces es posible procesar ambas secuencias por modelos diferentes:

- Un **encoder** procesa S_{contexto} y genera una representación contextual para S_{contexto} .
- Un **decoder** genera autorregresivamente la secuencia $S \in \mathcal{V}^*$ utilizando la representación contextual obtenida con el encoder.

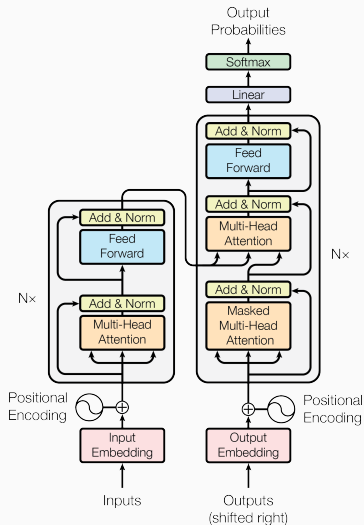


El modelo **encoder-decoder**, también llamado **seq2seq**, fue modelado originalmente usando RNNs. Sin embargo, también se puede utilizar una arquitectura **Transformer**:

- Un encoder tipo BERT procesa todo el contexto de entrada S_{contexto} .
- Un decoder tipo GPT genera la secuencia S utilizando la representación contextual obtenida con el encoder.
- Para esto, el decoder agrega un bloque de **atención cruzada** hacia las representaciones contextuales del encoder.

Arquitectura Transformer

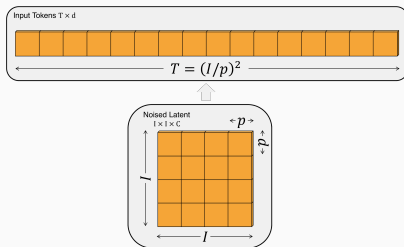
- GPT y BERT son arquitecturas derivadas del Transformer original.
- Hoy en día la mayoría de los modelos son de tipo GPT (**decoder-only**).
- Modelos encoder-decoder como **T5** siguen siendo ampliamente utilizados en modelos text-to-image.
- Los Transformers también tienen propiedades de aproximación universal y son Turing-completos.



Vision Transformer

Patchificación

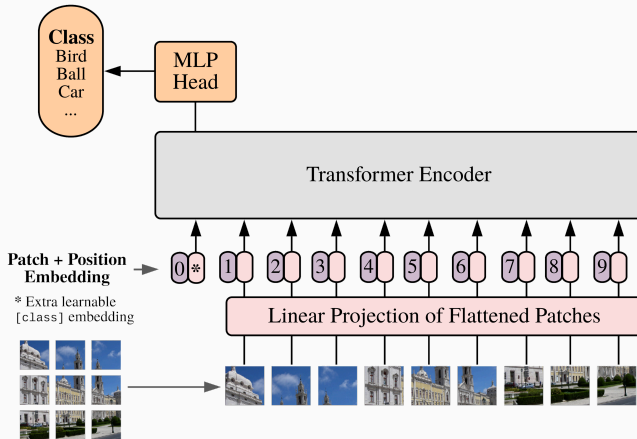
- BERT es un buen clasificador de secuencias, lo que motiva a transformar una imagen en una secuencia para usar BERT.
- La atención tiene costo cuadrático, por lo que no se podrían clasificar imágenes de alta resolución (de dimensión $I \times I \times C$) si se trata cada pixel como un token.
- **Vision Transformer (ViT)** propone dividir la imagen en **patches**, donde cada patch aplanado se trata como un token.



Teniendo esta representación secuencial de una imagen, se puede usar un modelo tipo BERT para clasificar la imagen:

- Al igual que en BERT, se agrega un token especial <CLS> cuyo embedding se utiliza para la clasificación.
- Una vez entrenado, se puede sustituir el cabezal de clasificación por otro y hacer fine tuning.
- ViT en general funciona mejor que una CNN pero requiere una gran cantidad de datos para su entrenamiento.

Clasificación

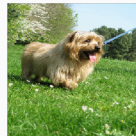


Visualización de la atención

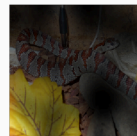
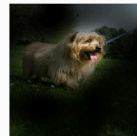
Dado que el ViT posee un mecanismo de atención, es posible visualizar la atención media que recibe cada patch desde el token <CLS>.

Esta visualización permite interpretar qué partes de la imagen son más relevantes para la clasificación.

Input



Attention



CLIP

Contrastive language-image pretraining

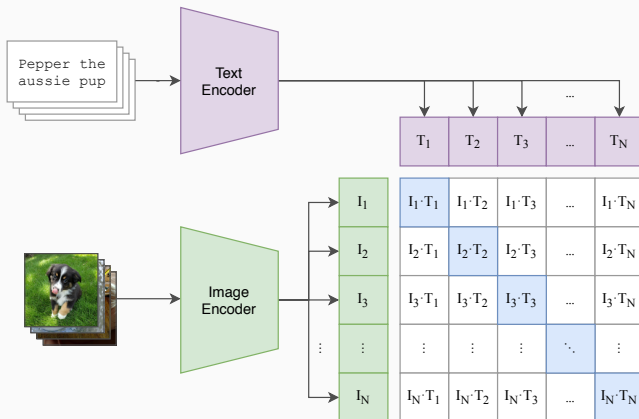
Se puede usar un image encoder (e.g. ViT) y un text encoder (e.g. BERT) para obtener representaciones de imágenes y texto en un mismo espacio de embedding.

Esto permite, por ejemplo, resolver las siguientes tareas:

- Dada una imagen, encontrar el texto más similar a la imagen.
- Dado un texto, encontrar la imagen más similar al texto.
- Realizar clustering de imágenes y texto en un mismo espacio de embedding.

Contrastive language-image pretraining

CLIP logra esto utilizando un **enfoque contrastivo**. En particular, CLIP no es un modelo generativo.



Entrenamiento

Si $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$ es un conjunto de pares de imágenes $x_n \in \mathbb{R}^{D_{\text{imagen}}}$ con sus respectivos textos descriptivos $t_n \in \mathcal{V}^*$:

- Se calculan los vectores de embedding $\text{ImageEncoder}(x_n) \in \mathbb{R}^{D_I}$ y $\text{TextEncoder}(t_n) \in \mathbb{R}^{D_T}$ para cada instancia $(x_n, t_n) \in \mathcal{D}$.
- Estos embeddings son proyectados a un espacio común, $\mathbb{R}^D$, para obtener nuevos embeddings $l_n, T_n \in \mathbb{R}^D$.
- De este modo, CLIP se entrena maximizando la **InfoNCE**:

$$L_{\text{InfoNCE}} = \sum_{i=1}^N \log \left(\frac{\exp(L_{ii})}{\sum_{j=1}^N \exp(L_{ij})} \right) + \sum_{j=1}^N \log \left(\frac{\exp(L_{jj})}{\sum_{i=1}^N \exp(L_{ij})} \right),$$

donde $L_{ij} = s_\theta \cdot \text{SimilitudCoseno}(l_i, T_j) \in \mathbb{R}$, con $s_\theta > 0$ un parámetro de temperatura (inversa) entrenable.

Recordar que la **similitud coseno** entre dos vectores $u, v \in \mathbb{R}^D$ se define como:

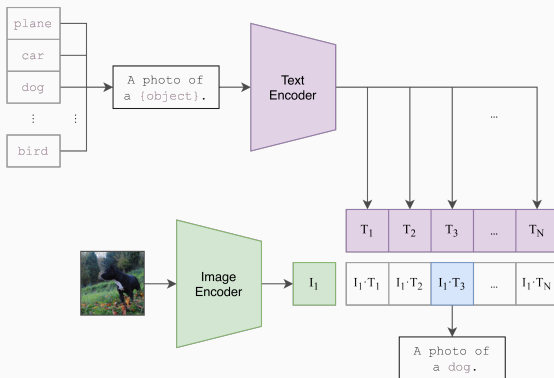
$$\text{SimilitudCoseno}(u, v) = \frac{\langle u, v \rangle}{\|u\| \|v\|} \in [-1, 1].$$

Por otro lado, la función objetivo se puede reescribir como:

$$L_{\text{InfoNCE}} = \sum_{i=1}^N \log \text{softmax}(\text{fila}_i(L))_i + \sum_{j=1}^N \log \text{softmax}(\text{columna}_j(L))_j$$

Además de los usos anteriores, se puede usar CLIP como un **open-vocabulary classifier** comparando una imagen con un conjunto de textos de la forma

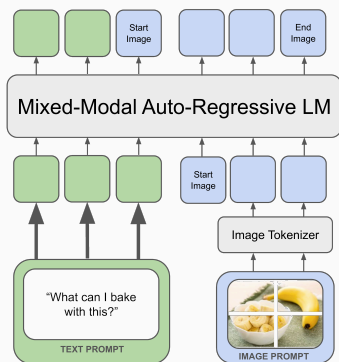
$S_{\langle \text{CLASE} \rangle} = \text{"una imagen de un/una } \langle \text{CLASE} \rangle \text{"}$



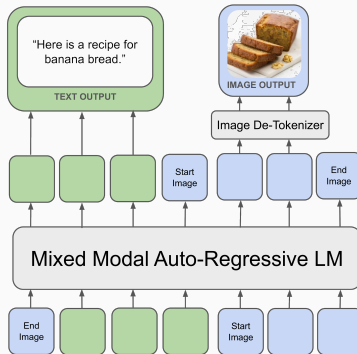
Chameleon

Chameleon

El modelo **Chameleon** utiliza la técnica de patchificación de imágenes para entrenar un modelo autorregresivo tipo GPT para generar texto e imágenes de manera conjunta.



(a) Mixed-Modal Pre-Training



(b) Mixed-Modal Generation

En la próxima clase:

- **LLMs:** chat templates, propiedades empíricas.
- **Agentes:** RAG, uso de herramientas, ReAct.

Modelos Generativos Profundos

Clase 8: Modelos autorregresivos y LLMs (parte IV)